

9 Corrected screens — light and dark, built from the CURRENT data, plus the acceptance checklist

The five screens from the audit (Turn 8), redrawn as they should ship — using the real workspace, run ids, function names and values from the CURRENT screenshots so the comparison is one-to-one. Each is rendered twice from the same markup: light, and dark via data-theme="dark" on the wrapper — no per-theme styling, tokens only. Agent sidebar (9a) · Configure (9b) · Run evidence (9c) · the run tab with graph, trace and the run-level inspector (9d) · Local runs (9e) · Trigger and Queue Configure (9g) · empty, app-down and no-Conductor states (9h) · the acceptance checklist per screen (9f). Everything here is CURRENT-scope except “Replay from start” / “Replay from here”, which remain the replay/fork primitive already marked CURRENT in Turn 7.

AGENT SIDEBAR

9a Agent — one header, six block classes, pinned composer

LIGHT

mBoss — Agentcodex ▾

Make the load plan carry the same trimmed customer ID used for duplicate detection.

Change 1/7: I'm fixing customer-ID canonicalization first. The load plan will now carry the same trimmed ID used for duplicate detection, preventing whitespace variants from becoming separate Airtable records.

Edit lib/airtableEtl.test.tsDONE

Edit lib/airtableEtl...+1-1APPLIED

142 srcAccountType: 'Enterprise*** ',

143 srcAddress: '12 Rúa Íris,, Apt 4;

144- srcCustomerId: 'C-100',

144+ srcCustomerId: ' C-100 ',

145 srcEmail: 'josé@example.com',

KeepUndo

Validate source and destin...RUNNING

Edit the graph, scaffold a lib fn, or ask why...

+ agent: codex ▾Stop

DARK · data-theme="dark"

mBoss — Agentcodex ▾

Make the load plan carry the same trimmed customer ID used for duplicate detection.

Change 1/7: I'm fixing customer-ID canonicalization first. The load plan will now carry the same trimmed ID used for duplicate detection, preventing whitespace variants from becoming separate Airtable records.

Edit lib/airtableEtl.test.tsDONE

Edit lib/airtableEtl...+1-1APPLIED

142 srcAccountType: 'Enterprise*** ',

143 srcAddress: '12 Rúa Íris,, Apt 4;

144- srcCustomerId: 'C-100',

144+ srcCustomerId: ' C-100 ',

145 srcEmail: 'josé@example.com',

KeepUndo

Validate source and destin...RUNNING

Edit the graph, scaffold a lib fn, or ask why...

+ agent: codex ▾Stop

AGENT Same transcript as the CURRENT screenshot: the “Change 1/7” prose, the airtableEtl.test.ts edit, the uniqueness

validation now a tool row in flight. The composer shows the working state with Stop in place of Send.

NODE INSPECTOR

9b Configure — Extract source records

LIGHT

Extract source records

api call

ConfigureRun evidence

function

extractAirtableRecords ✓

() → AirtableExtract

lib · matched by signature · 3 incompatible hidden · click to change

request

serviceAirtable

retry policy · configured

max attempts5

interval1 s

backoff2×

duration covers every try DBOS made · the ledger records one row

Open function

Ask agent

DARK · data-theme="dark"

Extract source records

api call

ConfigureRun evidence

function

extractAirtableRecords ✓

() → AirtableExtract

lib · matched by signature · 3 incompatible hidden · click to change

request

serviceAirtable

retry policy · configured

max attempts5

interval1 s

backoff2×

duration covers every try DBOS made · the ledger records one row

Open function

Ask agent

CONFIGURE Values from the CURRENT inspector: extractAirtableRecords, Airtable, 5 / 1 s / 2×. Rows are transparent at rest; the function picker is collapsed to its selection.

9c Run evidence — Upsert destination records

LIGHT

Upsert destination records

api call ✓ done · #2

ConfigureRun evidence

loadAirtableRecords ✓

AirtableRows → LoadResult

started18:24:37.822

completed18:24:38.873

duration1.1 s

retry policy · configured

max 5 · interval 1 s · backoff 2×

output · recorded result

INLINE { "extracted": 15, "created": 15, "updated": 0 }

Open function

Replay from hereAsk agent

recorded result · reused on recovery and on replay

EVIDENCE The CURRENT right-rail node card, compressed into the inspector: 24-hour times, the recorded result unwrapped from its superjson envelope, three actions in one row.

DARK · data-theme="dark"

Upsert destination records

api call ✓ done · #2

ConfigureRun evidence

loadAirtableRecords ✓

AirtableRows → LoadResult

started18:24:37.822

completed18:24:38.873

duration1.1 s

retry policy · configured

max 5 · interval 1 s · backoff 2×

output · recorded result

INLINE { "extracted": 15, "created": 15, "updated": 0 }

Open function

Replay from hereAsk agent

recorded result · reused on recovery and on replay

RUN TAB

9d Run tab — graph beside trace, run-level inspector when nothing is selected

LIGHT

run #7089 · airtable_etl · done · 1.6 s

GraphTrace

airtable_etl run ✓ done · #7089 1.6 s

ETL requested

trigger · on request

✓

↓

Extract source re...

f extractAirt...

✓

↓

Normalize and m...

f transformAi...

✓

↓

Upsert destinatio...

f loadAirtabl...

✓

workflow as saved · revision 4

trace

extract_records0.4 s

15 records

transform_recor...0.1 s

15 normalized · 0 rejected

load_records1.1 s

15 created · 0 updated

select a row or a node →
Run evidence in the inspector · DBOS-owned rows expand under their node

workflow input · as recorded

INLINE { "requestId": "airtable-etl-006" }

dbos.workflow_status

7089cd29-881b-workflow_uuid4319-a16d-1af70cc1e9a7

statusSUCCESS

recovery_attempts1

executor_idlocal

application_version1.0.0

The recovery ledger – your workflow is just rows in Postgres.

Replay from start

Ask agent

DARK · data-theme="dark"

run #7089 · airtable_etl · done · 1.6 s

Graph

Trace

ETL requested

trigger · on request

✓

↓

Extract source re...

f extractAirt...

✓

↓

Normalize and m...

f transformAi...

✓

↓

Upsert destinatio...

f loadAirtabl...

✓

workflow as saved · revision 4

trace

extract_records0.4 s

15 records

transform_recor...0.1 s

15 normalized · 0 rejected

load_records1.1 s

15 created · 0 updated

select a row or a node →
Run evidence in the inspector · DBOS-owned rows expand under their node

airtable_etl run ✓ done · #7089 1.6 s

workflow input · as recorded

INLINE { "requestId": "airtable-etl-006" }

dbos.workflow_status

7089cd29-881b-workflow_uuid4319-a16d-1af70cc1e9a7

statusSUCCESS

recovery_attempts1

executor_idlocal

application_version1.0.0

The recovery ledger – your workflow is just rows in Postgres.

Replay from start

Ask agent



RUN TAB The run tab is header + canvas + trace, nothing else. The trigger node is present; edges read done in green; no ports. The right card is what the sidebar inspector shows before a node is picked — workflow input and the ledger, with the raw SUCCESS kept there.

RUNS SIDEBAR

9e Local runs — six runs in one screen

LIGHT

Local runs

hello-world-test · this workspace

Run

Debug run

postgres :5432 · app :3000

All 6

Active 0

Failed 0

#7089

✓ done · 18:24 · 1.6 s · 3 step

Open on canvas

Replay from here

Ask agent

#4b56

✓ done · 18:17 · 1.5 s · 3 step

#b6e3

✓ done · 18:17 · 1.6 s · 3 step

#7245

✓ done · 18:16 · 1.9 s · 3 step

#4126

✓ done · 16:51 · 1.9 s · 3 step

#25c1

✓ done · 16:50 · 2.3 s · 3 step

local only · projected from dbos.workflow_status + dbos.operation_outputs

DARK · data-theme="dark"

Local runs

hello-world-test · this workspace

Run

Debug run

postgres :5432 · app :3000

All 6

Active 0

Failed 0

#7089

✓ done · 18:24 · 1.6 s · 3 step

Open on canvas

Replay from here

Ask agent

#4b56

✓ done · 18:17 · 1.5 s · 3 step

#b6e3

✓ done · 18:17 · 1.6 s · 3 step

#7245

✓ done · 18:16 · 1.9 s · 3 step

#4126

✓ done · 16:51 · 1.9 s · 3 step

#25c1

✓ done · 16:50 · 2.3 s · 3 step

local only · projected from dbos.workflow_status + dbos.operation_outputs

RUNS All six CURRENT runs, one line each; the newest is selected and carries the actions. Filter counts come from the same query as the list.

MORE CONFIGURE INSPECTORS

9g Configure — Trigger (owns no *f*) and Queue (NEXT)

LIGHT



ETL requested trigger

Configure

Run evidence

starts on

kind

on request

workflow

airtable_etl

input type

EtlRequest

a Trigger owns no *f* – it names the workflow and its input type · DBOS starts the workflow with that input

input · sample for Run

INLINE { "requestId": "airtable-etl-006" }

used by Run / Debug run only · not part of the saved workflow

Run with this input

Ask agent

Index documents queue

NEXT

Configure

Run evidence

handler

f indexDocument

Document → IndexResult

queue policy · registration

queue document-index

global concurrency

8

worker concurrency

2

rate limit

100 / 60 s

partitioning

off

global = across processes · worker = per process

enqueue policy · per item

priority unset

delay

none

deduplication

documentId

partitioned queues cannot deduplicate – turning partitioning on disables this field

Open function

Ask agent

DARK · data-theme="dark"

ETL requested trigger

Configure

Run evidence

starts on

kind

on request

workflow

airtable_etl

input type

EtlRequest

Index documents queue

NEXT

Configure

Run evidence

handler

f indexDocument

Document → IndexResult

queue policy · registration

queue document-index

a Trigger owns no *f* – it names the workflow and its input type · DBOS starts the workflow with that input

input · sample for Run

```
INLINE { "requestId":  
  "airtable-etl-006" }
```

used by Run / Debug run only
· not part of the saved workflow

Run with this input

Ask agent

global
concurrency 8

worker
concurrency 2

rate limit 100 / 60 s

partitioning off

global = across processes ·
worker = per process

enqueue policy · per item

priority unset

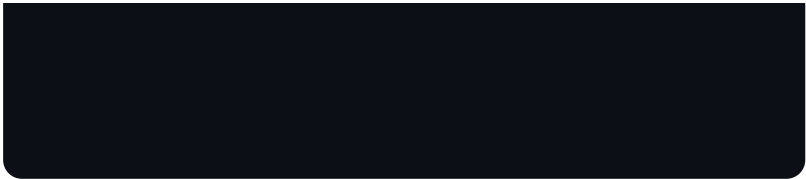
delay none

deduplication documentId

partitioned queues cannot
deduplicate – turning
partitioning on disables this
field

Open function

Ask agent



CONFIGURE Trigger has no function: it names the workflow, its input type, and a sample input for local Run — the sample is not saved into the workflow. Queue mirrors GI with the same PropertyRow / SectionLabel / FieldHint recipe; the registration vs per-item split is the mechanism, so it stays as two labelled groups.

EMPTY · DOWN · NO-CONDUCTOR STATES

9h Empty and boundary states — the panel says the fact and one way out

LIGHT

Local runs

hello-world-test · this workspace

Run

Debug run

postgres :5432 · app :3000

No runs recorded yet

Start the app and set a workflow going. Runs appear here as DBOS records them.

Run airtable_etl

local only · projected from dbos.workflow_status + dbos.operation_outputs

Local runs

hello-world-test · this workspace

postgres postgres:17 · :5432

app dbos start · :3000

The app is not running

Runs need the local DBOS app. Start it, then run a workflow.

Inspector airtable_etl.workflow

Pick a block to set what it does

Select a node on the canvas, or a run to see what it recorded.

production

Not connected to DBOS Conductor

mBoss local development is unaffected. Conductor manages deployed applications in production.

Learn about Conductor

deployment context only · never inside the local run loop

start app

DARK · data-theme="dark"

Local runs

hello-world-test · this workspace

Run

Debug run

postgres :5432 · app :3000

No runs recorded yet

Start the app and set a workflow going. Runs appear here as DBOS records them.

Run airtable_etl

local only · projected from dbos.workflow_status + dbos.operation_outputs

Local runs

hello-world-test · this workspace

postgres postgres:17 · :5432

app dbos start · :3000

The app is not running

Runs need the local DBOS app. Start it, then run a workflow.

Start app

Inspector airtable_etl.workflow

Pick a block to set what it does

Select a node on the canvas, or a run to see what it recorded.

production

Not connected to DBOS Conductor

mBoss local development is unaffected. Conductor manages deployed applications in production.

Learn about Conductor

deployment context only · never inside the local run loop

STATES EmptyState carries all four: no runs yet, nothing selected, app down (error kind, one primary fix), and the Conductor boundary stated without an upsell. Copy is the product's own I10n voice: fact first, then the way out. In dark mode the semantic inks brighten; nothing else changes. Rendering these surfaced a token bug (derived tokens not redeclared for dark) — fixed here with an override and listed in 9f.

ACCEPTANCE

9f Acceptance checklist — one column per screen, every line testable

Global 8j

Agent sidebar 9a

Configure inspector 9b

- No `text-transform:uppercase` anywhere except 10px tracked state words (DONE · APPLIED · INLINE · ARTIFACT); none of those are clickable.
- Every clickable action is a `Button` (primary · outline · quiet), sentence case, no letter-spacing.
- Every label/value pair is a `PropertyRow`: label left, value right; no value with a caption under it.
- Timestamps render 24-hour HH:MM:SS.mmm; durations 48 ms / 1.1 s; no AM/PM string appears in the DOM.
- Run ids render as # + first 4 hex; the full UUID appears only in the ledger card and the tab title.
- UI state words are lowercase (done · running · waiting · recovering · failed); SUCCESS appears only inside the dbos.workflow_status card.
- superjson envelope never reaches the DOM: no `__dbos_serializer` string in any rendered value.
- Dark mode: toggling `data-theme="dark"` on the webview root
- One header row: "mBoss — Agent" + quiet mono agent picker; the strings "MBoss" and "AGENT" do not appear.
- Transcript uses only the six block components: `UserMessage`, `AssistantProse`, `ToolEventRow`, `FileDiff`, `PermissionRow`, `Dagnostic`.
- `ToolEventRow` shows verb + concrete target (file path or run id), never "Editing files".
- `FileDiff`: workspace-relative path, +N -N counts, status word, single gutter column, ± column, common indentation stripped, no hunk-marker rows, lines never wrap.
- Keep / Undo are quiet Buttons in the `FileDiff` footer.
- `AssistantProse` renders inline markdown (bold, code, lists); literal `**` never shows.
- `Composer`: `resize:none`; height = content, min 3 lines, max 40% of panel (clamped 160–320px); attach / agent / Send-or-Stop in its meta row; Stop uses fail-tint, not amber.
- `Composer` is pinned to the panel bottom; transcript scrolls independently; no other
- Header = node name (600) + kind word (faint mono); no "NODE INSPECTOR" string.
- Tabs component "Configure | Run evidence", sentence case, brand underline.
- No bordered inputs at rest; `PropertyRow` values gain input chrome only on focus.
- takes / produces are not separate fields; the signature line of `LibFunctionItem` carries them.
- Exactly one `LibFunctionItem` visible at rest; the picker list, "incompatible hidden" line and "New function..." appear only while picking.
- `SectionLabels` group rows: "request", "retry policy · configured"; labels are single-line nouns, units live in values ("1 s", "2x").
- A `FieldHint` follows each policy group with the mechanism note.
- Actions row: "Open function" (brand outline) · "Ask agent" (quiet).
- Trigger inspector shows no function picker; it shows kind, workflow, input type

recolors every screen with no hard-coded hex left (audit: grep for #fff/#000 in component styles).

- Fonts: Albert Sans for language, Spline Sans Mono only for machine evidence; base 13px; no element below 10px.
- Design-system token bug: the derived tokens (--diff-add-bg, --diff-del-bg, --ink-faint, --hairline-strong, --ink-soft, --surface-ghost, --edge-done, --brand-ring, --brand-hover) are declared only in :root, so they resolve against light --surface/--ink and leak into dark mode — the diff rows go unreadable. Redecclare every derived token inside [data-theme="dark"] in tokens/colors.css.

Run evidence inspector 9c

- Subject is the selected node/operation; header keeps name + kind and adds "✓ done · #N" in state color.
- Order: LibFunctionItem → started / completed / duration PropertyRows → "retry policy · configured" one-liner → "output · recorded result" InlineValue or ArtifactRef → actions → FieldHint.
- Failed step: fail-tint error block with error name, then "Open function" primary + "Replay from here"; message states fact then way out.
- Workflow input, status, recovery_attempts, executor_id, application_version do not render in the node inspector.
- Actions: "Open function" · "Replay from here" · "Ask agent"; no "Open run".
- Footer FieldHint: "recorded result · reused on recovery and on replay".

view renders beneath it.

- 12px right padding keeps the scrollbar clear of content.

Run tab (Graph · Trace) 9d

- Header is one row ≤ 44px: StatusGlyph · mono "run #id · workflow · state · duration" · Tabs right · quiet ⌂.
- No breadcrumb, no "QUIET · REFRESH TO

and a sample input marked "used by Run only" (9g).

- Queue inspector has two labelled groups (registration · per item); deduplication disables when partitioning is on (9g).

Runs sidebar 9e

- Header "Local runs" + mono "workspace · this workspace"; Run (primary) / Debug run buttons; mono ports line.
- No "THIS SESSION" card; the newest run is the top RunHistoryItem, selected.
- Filters are Tabs "All N · Active N · Failed N"; no "Recovered" filter, no segmented bordered buttons.
- Each run is one 34px RunHistoryItem: 3px state rail · #id · workflow · summary; no large ✓ glyph; no dashed underline.
- Selected item expands with lineage line (when a replay exists) and "Open on canvas · Replay from here · Ask agent".
- Times 24-hour; today = time only, older = "Sep 11 18:24".
- One footer FieldHint line; Conductor sentence lives in the no-Conductor empty state (7m), not the footer.
- Six runs fit in ≤ 260px of list height.
- Empty / app-down / no-Conductor states use EmptyState with at most one action; no "upgrade" language (9h).

- no breadcrumbs, no "GO" button, no "CHECK", no 20px SUCCESS heading.
- ☐ Graph starts with the Trigger node; no edge originates off-node.
- ☐ No port dots rendered; edges colored by state (done = --edge-done, pending = hairline, live = dash-flow).
- ☐ Graph is anchored top-left with 22/26px padding; canvas $\approx$ 62% width, trace column the rest (or trace as a tab under 900px).
- ☐ Exactly one panel is mounted at a time; switching to Trace removes every graph node from the DOM (assert `querySelectorAll('[data-node]').length === 0`).
- ☐ Trace is one ExecutionTrace: glyph spine · mono op name · duration right · one detail line; SDK rows nest under their node. No STEPS chain, no Gantt, no operation_outputs table.
- ☐ Trace list max-width $\approx$ 640px inside a tab; selecting a row opens Run evidence in the sidebar inspector.
- ☐ Run-level inspector (nothing selected) holds workflow input + dbos.workflow_status ledger card + "Replay from start".

ACCEPTANCE Written so each line is a yes/no a reviewer or a test can answer against the running extension. Global lines first — they clear most per-screen items.

Try next: "Branch / Loop / Approval Configure inspectors" · "the permission request and diagnostic blocks in dark mode" · "keyboard map for the canvas"

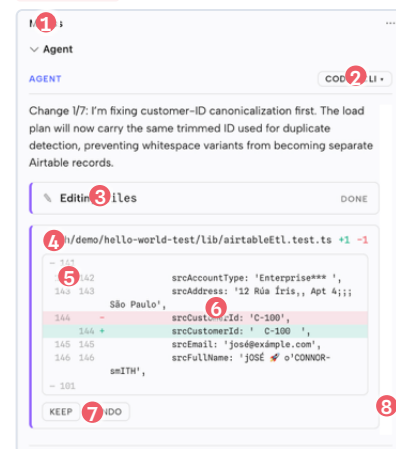
8 Implementation audit — CURRENT build vs FUTURE mockups, every gap called out for the next plan

Claude Code shipped the information architecture but not the Signal surface. Every pair below puts the shipped screen (red badges = defect at that spot, amber = something the mockup has that the build lacks) beside the mockup it was meant to match, then lists what to change and which Signal component carries it. Agent sidebar diffs (8a) · composer sizing bugs (8b) · Configure inspector (8c) · Run evidence inspector (8d) · blocks palette (8e) · run Graph view (8f) · Trace view overlap bugs (8g) · Runs sidebar (8h) · target layout of the run tab (8i) · the cross-cutting rules that explain most of the drift and a CURRENT-element → Signal-component map (8j). Root causes repeat: ALL CAPS used as voice, bordered caps pills instead of **Button**, value-above-label rows, full UUIDs as identifiers, 12-hour timestamps, raw `dbos` status words in the UI layer, port dots on the board, and two representations rendered where one was drawn.

AGENT SIDEBAR

8a Agent diffs — six block classes, one card per edit

CURRENT



FUTURE · 7n



one header row · tool rows
name their target · Edit
header IS the diff header ·
Keep / Undo as quiet text
buttons inside the card

- 1 Three headers for one panel — "mBoss" (wrong casing), a collapsible "Agent" section, then a blue caps "AGENT" label. → one row: "mBoss — Agent" at 600 weight; product name is always lowercase-m mBoss.
- 2 Agent picker as bordered caps pill — "CODEX CLI". → quiet mono text "codex" in --ink-muted, no border; lowercase machine name as the agent reports it. `Composer agent=`
- 3 Tool row is generic — "Editing files · DONE" with a pencil glyph and mono "files". → bold verb + mono target: "Edit lib/airtableEtl.test.ts"; no glyph; DONE stays as the 10px state word. `ToolEventRow`
- 4 Left-truncated absolute path — "...ash/demo/hello-world-test/lib/...". → workspace-relative path, truncate the middle only if needed: "lib/airtableEtl.test.ts".
- 5 Two-column old/new gutter and hunk-marker rows ("... 141", "142 142", "... 101"). → one gutter number, one ± sign column, tinted add/remove rows, no hunk rows; max 3 context lines each side. `FileDiff`
- 6 Source indentation preserved verbatim so code starts mid-card and wraps. → strip common leading whitespace; `white-space:pre` with horizontal scroll, never wrap diff lines.
- 7 KEEP / UNDO as bordered caps buttons in a separate zone. → "Keep" (brand quiet) · "Undo" (quiet) in the card footer, sentence case. `Button variant=quiet`
- 8 Nested cards + scrollbar overlap — diff table inside a card inside a purple-railed card; the panel scrollbar sits on top of content. → one card: header (verb · path · +1-1 · APPLIED) → lines → footer; 12px right

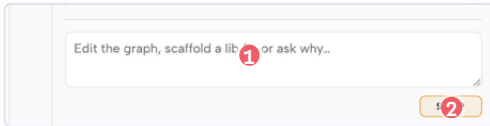
padding clears the scrollbar.

- + **Missing +/- count and APPLIED state on the edit header**; missing user-message bubble style (brand tint, full-width, no avatar) and the post-edit assistant line with next actions ("Replay from here" · "Undo edit"). `UserMessage` · `AssistantProse`

AGENT Six block classes only: user message, assistant prose, tool row, file edit, permission request, diagnostic. The build invented a seventh (a card wrapping a card) and lost the verb-plus-target grammar that makes tool rows scannable.

8b Composer — resize bugs and the Stop control

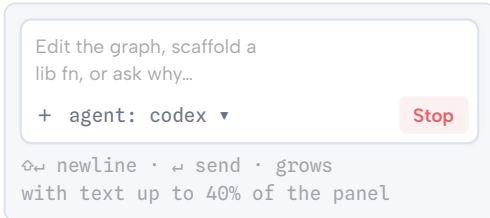
BUG · max size still too small



BUG · can be shrunk until unusable



TARGET · Composer while the agent works



- 1 **Maximum height $\approx$ 2 lines** — a long prompt scrolls inside a slot the height of a chip while the panel above sits empty. → `auto-grow with content: min 3 lines ( $\approx$  72 px incl. padding), max 40% of the sidebar height (clamp 160–320 px); overflow scrolls only past the max.`
 - 2 **STOP as an amber caps pill outside the composer** — floats below the field, right-aligned, unrelated to the input. → "Stop" lives inside the composer meta row, trailing edge, fail-tint background + --fail text, replaces the Send arrow while working. Stop is destructive-ish, so red family, not amber. `Composer` working onStop
 - 3 **Manual resize handle lets the field collapse to $\approx$ 28 px**, clipping the placeholder mid-glyph. → `resize:none`; drop the native grip entirely. Height is content-driven, never user-dragged. If a drag affordance is wanted, it belongs on the sidebar view splitter, not the textarea.
 - 4 **Raw markdown in assistant prose** — "***Validating source and destination uniqueness***" rendered with literal asterisks, in muted grey with a left rail. → render inline markdown (bold, code, lists) in `AssistantProse`; a streaming status line uses the tool-row shape ("Validate ... · RUNNING"), not prose.
 - 5 **Composer is not pinned** — a "Runs" section header (with a red square icon) scrolls up underneath it. → Agent view = scrolling transcript + composer pinned to the panel bottom (`flex column, transcript flex:1;overflow:auto`). Runs is a sibling view with its own header, never stacked under the composer.
- + **Meta row missing** — attach "+", "agent: codex ▾", model picker, and Send "↑" (brand when text present, `surface-2` when empty). Focus = brand border + focus ring; rest = hairline + --elev-1. Hint line in faint mono beneath.

COMPOSER Both bugs share one cause: height was handed to the browser's resize grip instead of the content. The Composer component already owns the meta row and the Stop state — mount it rather than a bare textarea plus a separate button.

NODE INSPECTOR

8c Configure — property rows, not a form

CURRENT · API call

FUTURE · Turn 6 Queue configure

- 1 **Tabs in tracked caps** — "CONFIGURE · RUN EVIDENCE". → "Configure" / "Run evidence", 13px sentence case, 2px brand underline on the active tab. `Tabs`
- 2 **"NODE INSPECTOR · API CALL" blue caps label** stands in for a

CONFIGURE

RUN EVIDENCE

NODE INSPECT

API CALL

title

Extract source records

takes

produces

AirtableExtract

function

extractAirtableRecords

open f

/LIB · MATCHED BY SIGNATURE

f extractAirtableRecords

() → AirtableExtract

3 incompatible functions hidden · show

New function...

service

Airtable

attempts

5

first retry after, in

1

seconds

1

backoff, times

2

Index documents

queue

Configure

Run evidence

handler

f indexDocument

Document → IndexResult

queue policy · registration

queue

document-index

global concurrency

8

worker concurrency

2

rate limit

100 / 60 sec

partitioning

off

global = across processes · worker = per process · advanced: partition limits, min polling interval

enqueue policy · per item

priority

unset

delay

none

deduplication

documentId

partitioned queues cannot deduplicate · turning partitioning on disables this field

node name + faint kind word as the header · Tabs · "handler" → one LibFunctionItem · section labels group policy rows · FieldHint carries the mechanism

- title. → header = node name in 600 weight ("Extract source records") + kind word in --ink-faint ("api call"), above the tabs. On a run it also carries the state word right-aligned ("✓ done · #2").
- 3 title / takes / produces as bordered inputs; "takes" sits empty with a full input frame. → PropertyRow: label left in --ink-muted, mono value right, hairline between rows; input chrome appears only on focus. takes/produces are read from the bound function's signature — show them once, in the LibFunctionItem signature line, not as duplicate fields.
- 4 Function row is a dropdown-looking text line ("extractAirtableRecords · open f") plus a second card for the same function. → label "function" (or "handler" for Queue) then one selected LibFunctionItem (f name, signature, ✓). Click it to open the picker; "Open function" is a quiet button in the actions row, not an inline "open f".
- 5 Picker always expanded — "/LIB · MATCHED BY SIGNATURE", "3 incompatible functions hidden · show", "New function..." permanently occupy the panel. → collapsed at rest; the list, the incompatible-hidden line and "New function..." appear only while picking. Label reads "lib · matched by signature" in the 11px section-label style. SectionLabel
- 6 Retry policy fields have no heading and use input frames. → SectionLabel "retry policy · configured" then PropertyRows: "max attempts 5", "interval 1 s", "backoff 2x"; under them a FieldHint: "duration covers every try DBOS made · the ledger records one row".
- 7 Labels wrap to two lines ("first retry after, in seconds"). → short noun labels; units live in the value ("1 s"), never in the label.
- + Missing: the "service" row belongs under a "request" section label; a FieldHint after the function explaining the semantics of the kind (API call: "one durable step · retried by DBOS on failure"); 12/14px panel padding consistent with the Runs view.

CONFIGURE Hierarchy through space and type weight, not boxes. Every bordered input at rest is a box the mockup does not have; every caps label is a heading the mockup writes in sentence case.

8d Run evidence — step-level facts, workflow-level facts elsewhere

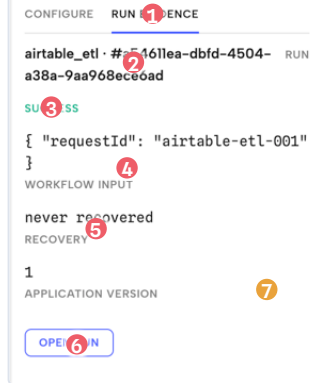
CURRENT

FUTURE · 7c

1 Caps tabs, same fix as 8c. Tabs

2 Wrong subject — the tab shows the workflow ("airtable_etl · #a5461lea-..." in bold, wrapping over two lines) instead of the selected step. → Run evidence is per node: header keeps the node name + kind; the run id appears once, short, in the state line ("✓ done · #a546").

3 "SUCCESS" as a large green caps heading — that is the raw dbos.workflow_status.status value. → UI state words are lowercase: "done", 12px green, with the ✓ glyph; the raw SUCCESS string appears only inside the ledger



card. StatusGlyph

- 4 **Value-above-label pattern** — the JSON is printed first and "WORKFLOW INPUT" is captioned under it, three times over. → label first, always: `PropertyRow` for started / completed / duration; `SectionLabel` "output · recorded result" followed by an `InlineValue` (INLINE tag) or `ArtifactRef` (ARTIFACT tag + open).
- 5 **"never recovered" / "I" as free-standing values** under RECOVERY / APPLICATION VERSION. → these are workflow-level ledger fields: `recovery_attempts 1`, `application_version 1` as mono `PropertyRows` inside a run-level "dbos.workflow_status" card, shown when the run (not a node) is selected.
- 6 **OPEN RUN bordered caps button** — the run is already open. → actions for a step: "Open function" (brand outline) · "Replay from here" (quiet) · "Ask agent" (quiet); sentence case. Button
- 7 **Everything step-level is missing:** the selected `LibFunctionItem` with signature, started / completed / duration in 24-hour mono ("10:31:14.218"), "retry policy · configured" one-liner ("max 5 · interval 1 s · backoff 2×"), recorded output, and the footer `FieldHint` "recorded result · reused on recovery and on replay".

EVIDENCE Same panel, two clean modes. Configure answers "what will this node do"; Run evidence answers "what did this step record". Workflow input, status and recovery attempts are facts about the run, and belong at the run level (8i).

CANVAS

8e Blocks palette — icon tiles, one flat list

CURRENT

FUTURE · Turn 5 palette

- 1 **"BLOCKS" in caps.** → "blocks": 11px, weight 500, +0.05em tracking, `--ink-muted`. `SectionLabel`
- 2 **Four group headings (START / WORK / CONTROL / PEOPLE)** in caps mono. → no groups. One flat list in canonical order: Trigger, Step, Transaction, API call, Code step, Queue, Branch, Loop, Wait, Approval, Email. Ordering carries the grouping; headings do not.
- 3 **Items are full-width grey boxes with no icons**, ~40px tall. → transparent rows: `NodeIconTile` (24px, `--surface-2` tint, Lucide 2px stroke) + name; hover shifts the row to `--surface-2`; ~30px row pitch; drag handle is the whole row.
- 4 **Queue present** — correct (NEXT scope, 61); the mockup predates it. → keep it; give it the Lucide `list-ordered` tile so it reads as a kind like the others.
- 5 **Label mismatch** — build says "Wait" / "Email", mockup says "Durable wait" / "Email send". → pick one string per kind and use it in the palette, the canvas kind word and the inspector header. Recommendation: match the mockup ("Durable wait", "Email send") — the qualifier explains the mechanism.
- 6 **Footer hint missing.** → `FieldHint` in faint mono: "drag onto the canvas · drop on an edge to splice".

ST 2

Trigger 3

WORK

Step

Transaction

API call

Code step

Queue 4

CONTROL

Branch

Loop

Wait 5

PEOPLE

Approval

Email 6

Trigger

Step

Transaction

API call

Code step

Branch

Loop

Durable wait

Approval

Email send

drag onto the canvas
· drop on an edge to splice

11px "blocks" label
· 24px icon tile +
13px name · ~30px
rows · mono footer
hint

PALETTE Kind is expressed through iconography, never color and never a box. The palette is the first place the ten icons appear; the canvas and inspector reuse the same tiles.

RUN VIEW — GRAPH · TRACE

8f Graph view — one header row, a canvas that starts where the eye does

CURRENT

run #5218 · refundApproval · waiting

Refund requested
method: POST /refunds

Load purchase
getPurchase

Evaluate refund
refundPolicy

Request approval
WAITING: 02:54 durable

Refund payment
refundPayment

7b

Graph / Trace — two representations of the same execution

Graph

Trace

trace

refund_requested 12 ms

get_purchase 48 ms

refund_policy 9 ms

request_approval

waiting 2 * 14 * 5 * 10ms * 2

requestApproval - 2 durable operations

send approval request

durable receive

graph answers "where is it in my workflow" · trace answers "what durable operations happened, in order" · one node may map to several durable operations

run #5218 · refundApproval · waiting

Refund requested
method: POST /refunds

Load purchase
getPurchase

Evaluate refund
refundPolicy

Request approval
WAITING: 02:54 durable

Refund payment
refundPayment

7b

Graph / Trace — two representations of the same execution

Graph

Trace

trace

refund_requested 12 ms

get_purchase 48 ms

refund_policy 9 ms

request_approval

waiting 2 * 14 * 5 * 10ms * 2

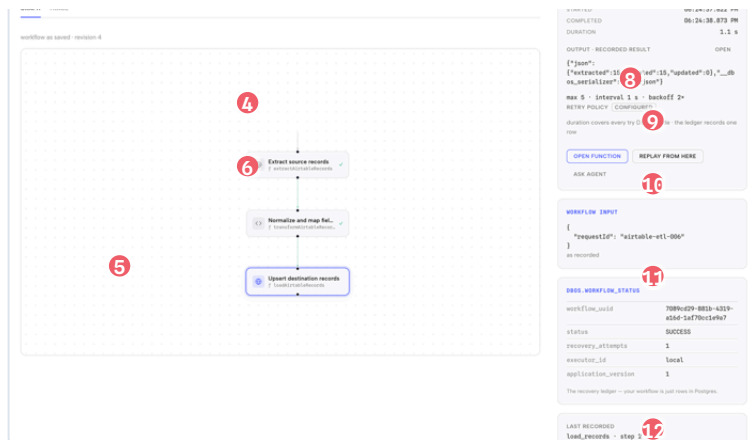
requestApproval - 2 durable operations

send approval request

durable receive

graph answers "where is it in my workflow" · trace answers "what durable operations happened, in order" · one node may map to several durable operations

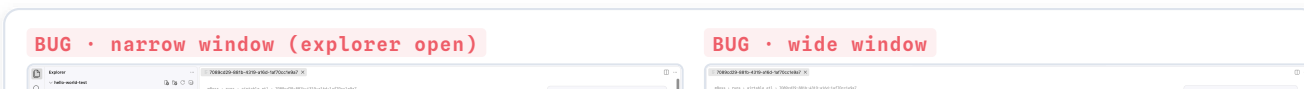
44px header: StatusGlyph · mono run line · Tabs right · graph left, trace right · nodes 230px, edges green when done, amber glow on the waiting node · no ports

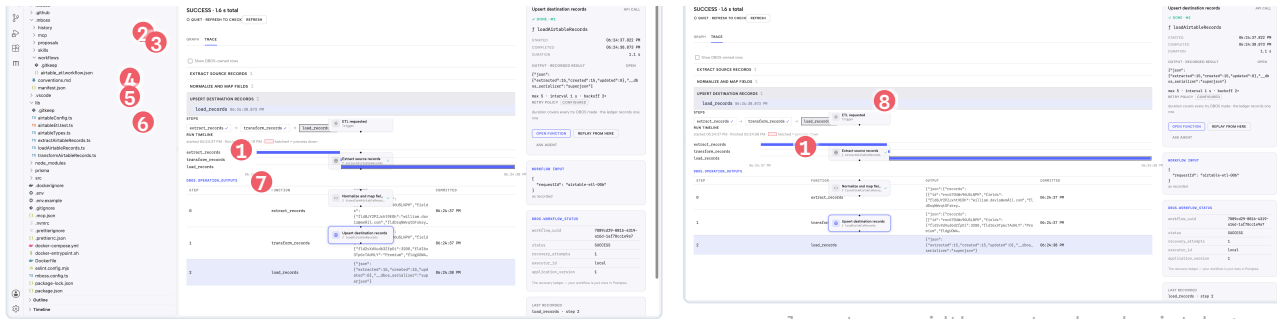


- 1 Header stack ~300px tall — mono breadcrumb with full UUID, then "SUCCESS · 1.6 s total" as a 20px heading. → one 44px row: StatusGlyph + mono "run #7089 · airtable_etl · done · 1.6 s". Breadcrumb goes to the VS Code tab title.
- 2 "O QUIET · REFRESH TO CHECK [REFRESH]" — caps status + caps button. → live runs poll; finished runs are static. If a manual refresh must exist it is a quiet "O" icon button at the header's right, no label.
- 3 GRAPH / TRACE caps tabs on their own row, plus "workflow as saved · revision 4" on another. → Tabs "Graph | Trace" right-aligned in the header row; the revision note becomes a FieldHint in the canvas corner.
- 4 Dangling edge, missing Trigger node — the first edge descends from nothing. → render the Trigger node ("ETL requested · trigger") as the first node; every graph starts at its trigger. WorkflowNode kind=trigger
- 5 Three nodes centred in an ocean of dots. → graph anchored top-left with 22/26px padding inside CanvasSurface; the canvas column is ~62% of the tab, the trace column the rest (see 8i). Pan/zoom, no auto-centering on load.
- 6 Black port dots on node tops and bottoms; edges are thin grey with a tiny green arrowhead. → ports never on the board. WorkflowEdge state=done: --edge-done green stroke + arrowhead; pending edges hairline grey; the live edge dash-flows.
- 7 Right rail: caps label rows (STARTED / COMPLETED / DURATION, right-aligned 12-hour values "06:24:37.822 PM"). → PropertyRow lowercase labels, mono 24-hour values "18:24:37.822", left-aligned value column.
- 8 "RETRY POLICY [CONFIGURED]" pill chip with the summary line above the label. → SectionLabel "retry policy · configured" then the mono one-liner; no chip.
- 9 OPEN FUNCTION / REPLAY FROM HERE / ASK AGENT as caps bordered pills. → "Open function" brand outline · "Replay from here" quiet · "Ask agent" quiet, one row, 8px gap. Button
- 10 WORKFLOW INPUT card under the node card. → run-level fact. Show it in the run-level inspector (nothing selected) as SectionLabel "workflow input · as recorded" + InlineValue; not stacked under every node.
- 11 DBOS.WORKFLOW_STATUS card — right idea (the ledger card from 7c / 71) but caps blue heading and stacked under the node. → run-level inspector; heading in mono lowercase "dbos.workflow_status"; keep the raw SUCCESS here and only here; footer stays: "The recovery ledger — your workflow is just rows in Postgres."
- 12 LAST RECORDED card duplicates the trace's last row. → drop; the trace and the node state already say it. Also: the recorded output shows the superjson envelope ({ "json": ..., "__dbos_serializer": "superjson" }) — unwrap before display, INLINE when ≤ ~120 chars, otherwise ARTIFACT + open.

GRAPH The canvas stays the primary mental model. The build spent a third of the viewport on chrome before the first node, and moved everything the mockup keeps in the sidebar inspector into a stack of caps-labelled cards.

8g Trace view — the graph bleeds through, and four traces where one was drawn





same overlap at any width → not a breakpoint bug;
the graph layer is still mounted under the Trace tab

- Graph nodes render on top of the trace** — the four node cards (with port dots and the selection halo) float over the trace at the same x/y they hold on the canvas. → tab panels are mutually exclusive: unmount (or `display:none`) the graph layer when Trace is active. Nodes are positioned inside their own `CanvasSurface` container, never in the tab's shared coordinate space. Re-render on tab switch; don't share one absolutely-positioned layer between panels.
- "Show DBOS-owned rows" checkbox** above the trace. → SDK-level rows group under their node and expand inline ("requestApproval · 2 durable operations ↳ send ↳ receive"); a global checkbox is not needed. If kept, it is a quiet toggle at the trace footer.
- Caps node-group headings with counts** ("EXTRACT SOURCE RECORDS 1"). → trace rows are the operations themselves: `StatusGlyph` `spine` · `mono` `extract_records` · duration right · one faint detail line. The node name is the row's detail, not a heading. `ExecutionTrace`
- "STEPS" pill chain** (extract ✓ → transform ✓ → load). → remove; it repeats the Graph tab.
- "RUN TIMELINE" Gantt** with a hatched "process down" legend. → remove from the trace; durations already sit on each row. A Gantt is at most an EXPLORATION for long-running runs and would be its own tab, not stacked here.
- "DBOS.OPERATION_OUTPUTS" table** (STEP · FUNCTION · OUTPUT · COMMITTED) with raw JSON wrapping in cells. → remove; output belongs in the step inspector when a trace row is selected. The trace row's detail line may show a short inline summary ("15 extracted").
- Serializer envelope shown raw** — `{"json": { "...", "__dbos_serializer": "superjson" }}`. → unwrap `superjson` before rendering anywhere in the UI; the raw envelope is visible only via "open" on an `ArtifactRef`.
- Full-bleed at wide widths** — bars and table stretch across 1900px for a 1.6 s run. → the trace list has `max-width ≈ 640px` inside the Trace tab (or lives in the right column beside the graph, per 8i). Selecting a row opens the same Run evidence inspector as selecting a node.

TRACE "Graph answers where; trace answers what happened, in order." The build answered "what happened" four times. Consolidate to one vertical trace and the overlap bug disappears with the layout that caused it.

RUNS SIDEBAR

8h Local runs — a dense IDE list, not a card feed

CURRENT

FUTURE · 7a

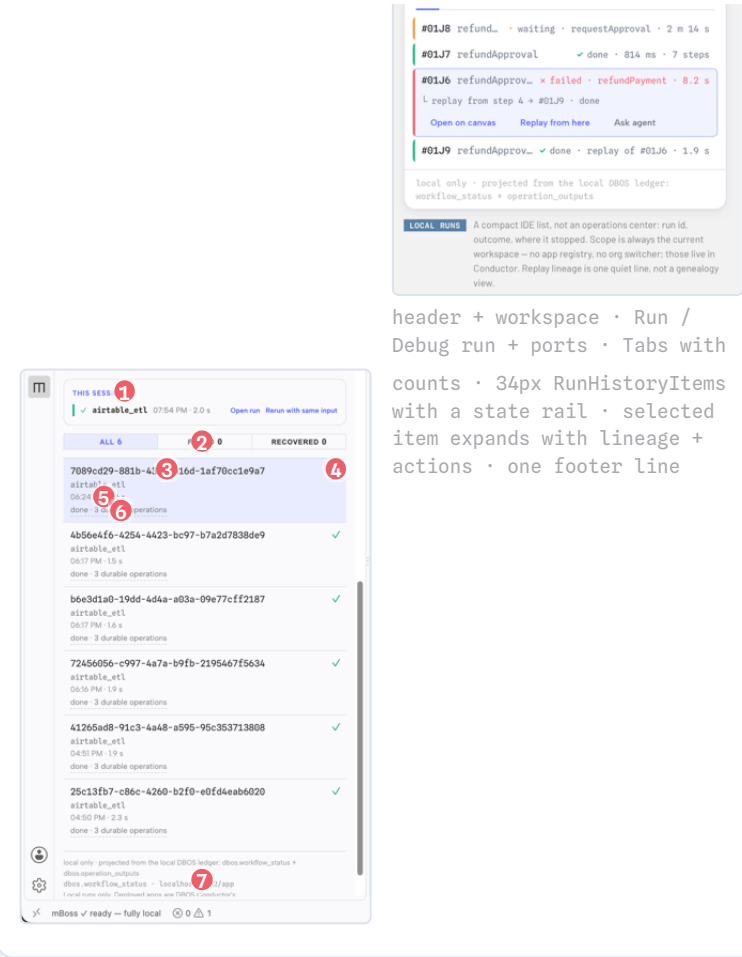
7a Local runs — this workspace, this app, this developer

Local runs
refund-service · this workspace

Run
Debug run
postgres :5432 · app :3000

All 4 Active 3 Failed 1

- "THIS SESSION" card** — caps label, green bar, its own Open run / Rerun links. → no session card. The just-started run is simply the top `RunHistoryItem`, selected; its actions ("Open on canvas" · "Replay from here" · "Ask agent") appear in the expanded item. Above the list: "Local runs" + mono "hello-world-test ·



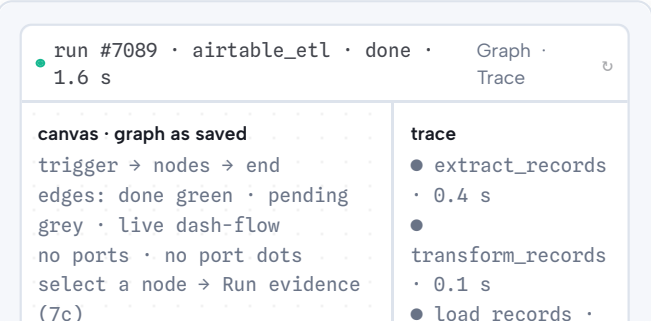
header + workspace · Run /
Debug run + ports · Tabs with
counts · 34px RunHistoryItems
with a state rail · selected
item expands with lineage +
actions · one footer line

- this workspace", then Run / Debug run Button s +
"postgres :5432 · app :3000".
- 2 Segmented caps filter buttons "ALL 6 | FAILED 0 |
RECOVERED 0", full-width bordered. → Tabs : "All 6 ·
Active 0 · Failed 0" sentence case, count in brand on
the active tab. "Recovered" is a state shown on the
item's rail, not a filter.
 - 3 Full 36-char UUID as the bold primary line. → short id
"#7089" (first 4 hex) in mono 600; full uuid only in the
ledger card and on hover title.
 - 4 Four-line items with a large ✓ at right, ~180px each; six
runs need 1100px. → one line, 34px: state rail (3px, --
ok/--warn/--fail) · #id · workflow mono · "done ·
1.6 s · 3 steps". State reads from the rail + word, no big
glyph. RunHistoryItem
 - 5 12-hour "06:24 PM". → 24-hour mono "18:24"; today's
runs show time only, older ones "Sep 11 18:24".
 - 6 "done · 3 durable operations" with a dashed underline
— reads as a link or tooltip trigger. → plain summary
text in the item's one line; no underline.
 - 7 Three overlapping footer lines, the last clipped ("Local
runs only. Deployed apps are DBOS Conductor's..."). →
one FieldHint: "local only · projected from
dbos.workflow_status + dbos.operation_outputs". The
Conductor sentence belongs to the empty/no-
Conductor state (7m), not the footer.
- + Missing: Run / Debug run buttons and the ports line;
the selected item's replay-lineage line (" L replay from
step 2 → #4b56 · done"); hover --surface-2 and
selected --brand-tint row states.

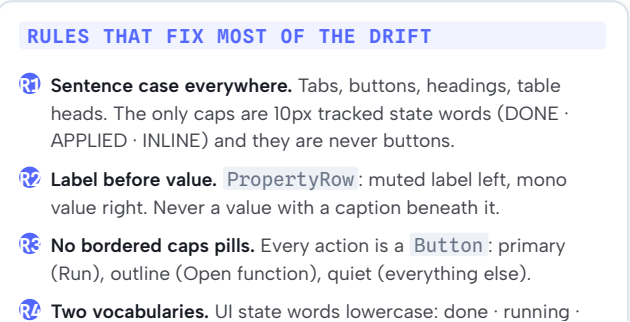
RUNS Run id, outcome, where it stopped — one line each. Density is the feature: a developer scans twenty runs without scrolling.

TARGET LAYOUT · RULES · COMPONENT MAP

8i Run tab — where each fact lives



8j Cross-cutting rules and the CURRENT → Signal component map



revision 4 · as saved

1.1 s
15 extracted ·
15 created
select a row →
same inspector

sidebar inspector · node selected
name + kind · ✓ done · #2
Configure | Run evidence
f card ·
started/completed/duration

sidebar inspector ·
nothing selected
run #7089 ·
airtable_etl
workflow input · as
recorded
dbos.workflow_status
ledger card
status SUCCESS ·
recovery_attempts 1
executor_id local
·
application_version
1
Replay from start ·
Ask agent

LAYOUT Node facts in the node inspector, run facts in the run inspector, and the trace beside the graph rather than under three other traces. Trace may also be a tab when the tab is narrower than ~900px — but only one panel is ever mounted.

waiting · recovering · failed. Raw ledger words (SUCCESS, PENDING, ERROR) appear only inside the dbos.workflow_status card.

- Identifiers.** Short id "#7089" everywhere; full UUID in the ledger card and tab title only. Paths workspace-relative.
- Time.** 24-hour mono, ms when it matters: "18:24:37.822"; durations "48 ms" · "1.1 s". No AM/PM.
- Values.** Unwrap the superjson envelope. ≤ ~120 chars → InlineValue (INLINE); longer → ArtifactRef (ARTIFACT · open).
- Board hygiene.** Ports and port dots never render; edges colour by state; nodes show icon + name + f only. One representation per view.
- Boxes are earned.** Inputs get chrome on focus; palette rows, property rows and trace rows are transparent at rest; hairlines separate, cards float only over the canvas.

CURRENT ELEMENT → SIGNAL COMPONENT

caps tab rows	core/Tabs
bordered caps buttons	core/Button primary · outline · quiet
bordered inputs at rest	inspector/PropertyRow (+ core/Input on focus)
caps section headings	core/SectionLabel
grey explanatory sentences	inspector/FieldHint
function dropdown + card	inspector/LibFunctionItem
SUCCESS heading · ✓ glyphs	runs/StatusGlyph
4-line run cards	runs/RunHistoryItem
steps chain · gantt · outputs table	runs/ExecutionTrace (one)
raw JSON blocks	data/InlineValue · data/ArtifactRef
palette grey boxes	canvas/NodeIconTile + row
dotted-edge nodes	canvas/WorkflowNode · WorkflowEdge · CanvasSurface
"Editing files" row	agent/ToolEventRow
nested diff cards	agent/FileDiff
textarea + STOP pill	core/Composer
markdown prose	agent/AssistantProse

RULES Nine rules cover roughly forty of the findings above. Fix the rules in shared components first, then walk the screens; most per-screen items fall out of R1-R3 alone.